

ASSETMATCH

Software Requirements Specification (SRS)

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a comprehensive architectural and functional overview of the ASSETMATCH project. ASSETMATCH is an AI-powered visual reverse image search engine designed to solve workflow bottlenecks in digital studios. It allows users to upload a reference image, converts the image into a mathematical vector representation, and instantly retrieves visually similar assets from a database using advanced similarity search techniques.

1.2 Scope

The system is a fully containerized full-stack web application consisting of three primary modules orchestrated via Docker Compose:

- **Frontend:** A responsive, interactive React UI built with Vite and TypeScript.
- **Backend API:** A high-performance RESTful API built with Python and FastAPI, handling file processing and database queries.
- **Machine Learning Engine:** An offline PyTorch module utilizing a pre-trained ResNet18 Convolutional Neural Network (CNN) to extract 512-dimensional feature vectors.
- **Database:** A PostgreSQL database extended with `pgvector` to store asset metadata and perform efficient similarity indexing.

1.3 Definitions & Acronyms

- **API:** Application Programming Interface
- **CNN:** Convolutional Neural Network (e.g., ResNet18)
- **pgvector:** An open-source vector similarity search extension for PostgreSQL.
- **Embedding:** A mathematical vector ($\mathbb{R}^{512}$ array) representing the visual features of an image.
- **Cosine Distance:** A metric used to measure how similar two vectors are, irrespective of their magnitude.

2. Overall Description

2.1 Product Perspective

ASSETMATCH operates as an independent, self-hosted system. The architecture relies on an isolated Docker network (`asset_network`) ensuring seamless communication between the UI on port 3000, the API on port 8000, and the database on port 5432. It is engineered to remove the reliance on file-naming conventions by introducing content-based image retrieval.

2.2 Product Functions

The primary functions of the system include:

- Automated database seeding and initialization of a vector schema upon startup.
- Real-time image tensor transformation and feature extraction.
- Vector similarity search utilizing cosine distance ($1 - \cos(\theta)$).
- Dynamic results filtering based on aspect ratio bounds, file extensions, and similarity thresholds.
- Local browser state management for upload history and drag-and-drop interface handling.

2.3 User Classes and Characteristics

- **End Users (Designers/Artists):** Require a visually intuitive, drag-and-drop interface with minimal latency. They are interested in visual results rather than the underlying ML processes.
- **System Administrators:** Require robust, containerized deployment, clear logging, and automated database initialization for easy server integration.

3. System Features (Functional Requirements)

3.1 Feature Vector Extraction (ML Engine)

Description: Upon receiving an uploaded image, the backend must convert it into a 512-dimensional float array.

Requirements:

- The system must accept binary image bytes (PNG, JPG, JPEG).
- The system must resize images to 256x256 and center-crop to 224x224.

- The system must normalize image tensors using standard ImageNet metrics (Mean: [0.485, 0.456, 0.406], Std: [0.229, 0.224, 0.225]).
- The system must process the tensor through a truncated ResNet18 model (evaluation mode, gradients disabled) to output a 1D vector.

3.2 Vector Similarity Search

Description: The database must find the closest matching assets to the reference vector.

Requirements:

- The query must utilize the `<=>` pgvector operator to calculate cosine distance.
- The system must convert distance into a similarity percentage (*Similarity = 1 - Distance*).
- The query must return a maximum of 6 matches ordered by highest similarity.

3.3 Advanced Dynamic Filtering

Description: The user can constrain the search results through the sidebar interface.

Requirements:

- **Orientation Filter:** Must filter SQL queries by aspect ratio (*width / height*). Horizontal (*aspect_ratio > 1.0*), Vertical (*aspect_ratio < 1.0*).
- **File Type Filter:** Must apply dynamic `ANY()` SQL clauses to filter by array of extensions (e.g., png, jpg).
- **Variety Threshold:** Must limit maximum cosine distance (Low = 0.15, Medium = 0.40, High = 0.80).

4. External Interface Requirements

4.1 User Interfaces

The frontend application must provide a modern, pastel-themed interface characterized by glassmorphism. It requires a split-pane layout with an interactive drag-and-drop upload zone on the left and a collapsible configuration sidebar on the right. Results must be displayed in a responsive 4-column grid (shrinking to 2 columns on mobile devices).

4.2 Software Interfaces

Component	Technology	Purpose
Database Schema	PostgreSQL + pgvector	Persistent storage of file metadata and vector embeddings.
Backend API	FastAPI + Uvicorn	Exposing HTTP endpoints (<code>/api/search</code> , <code>/api/download</code>).
ML Framework	PyTorch + torchvision	Executing the ResNet18 neural network inferences.
Frontend State	React + TypeScript	Managing DOM updates, session history, and filter states.

4.3 Communications Interfaces

All communication between the React frontend and FastAPI backend must happen via REST over HTTP. The backend must enforce CORS (Cross-Origin Resource Sharing) policies allowing requests exclusively from `http://localhost:3000` . File uploads must utilize `multipart/form-data` payloads.

5. Non-Functional Requirements

5.1 Performance Requirements

- The entire upload-to-result lifecycle should process in under 2 seconds (hardware dependent) without the necessity of GPU acceleration (running purely on CPU tensors).
- The UI must maintain 60 FPS scrolling and transition animations for the sidebar and carousel.

5.2 Reliability & Availability

- Docker container policies must be set to `restart: unless-stopped` to ensure maximum uptime in case of application crashes.
- The database initialization module must securely handle rollback procedures if vector generation fails during the automated seeding process.

5.3 Security Requirements

- The system must strictly validate incoming file MIME types, rejecting any payloads that do not match image signatures.
- Environment variables must be implemented for PostgreSQL credentials, preventing hardcoded secrets in the production `database.py` module.